Détection d'anomalies dans les transactions bancaires

Approche par clustering K-means

Rapport d'analyse et d'interprétation

Réalisé par :

d'ALMEIDA Ayité Hyacinthe Junior

AREGBA-TISSOU Ulrich

21 juin 2026

Table des matières

1 Contexte et objectif **3**

2 Données et environnement **3**

2.1 Le jeu de données . . . 3

2.2 Hypothèses et conventions de départ . . . 3

2.3 Environnement technique . . . 4

3 Méthodologie **4**

3.1 Pourquoi normaliser ? . . . 4

3.2 Détection des anomalies . . . 5

3.3 Choix automatique du seuil . . . 5

3.4 Réduction de dimensions par PCA . . . 7

4 Analyse exploratoire des données **8**

4.1 Qualité des données . . . 8

4.2 Des distributions très asymétriques . . . 8

4.3 Forte corrélation entre variables . . . 10

4.4 Premiers comportements repérés . . . 10

5 Résultats du clustering **11**

5.1 Méthode d'interprétation . . . 11

5.2 Profil des six clusters . . . 11

6 Analyse des anomalies détectées **13**

6.1 Combien d'anomalies ? . . . 14

6.2 À quels clusters appartiennent-elles ? . . . 14

6.3 Que révèlent-elles ? Un constat essentiel . . . 16

7 Interprétation critique **17**

7.1 Une détection aveugle aux incohérences logiques . . . 17

7.2 Les causes profondes . . . 18

7.3 Conséquence pour l'usage . . . 18

8 Recommandations **18**

9 Conclusion 19

10 Références 19

A Lancement du projet 20

B Script principal — main.py 20

C Analyse exploratoire — EDA.py 21

D Pipeline K-means — k_means_pipeline.py 25

1. Contexte et objectif

L'objectif de ce projet est d'identifier automatiquement des comportements inhabituels au sein d'un ensemble de 20 000 transactions bancaires, **sans disposer d'étiquettes préalables** indiquant quelles transactions sont normales ou frauduleuses. La démarche relève donc de l'**apprentissage non supervisé**.

Le principe est le suivant : la grande majorité des transactions suivent des schémas réguliers et se regroupent naturellement en familles de comportements semblables. L'algorithme *K-means* construit ces familles (les *clusters*) en rassemblant les transactions proches les unes des autres. Une transaction qui s'écarte fortement du centre de son propre groupe ne ressemble à aucun schéma connu : elle est alors considérée comme une **anomalie** potentielle, méritant une attention particulière.

2. Données et environnement

2.1 Le jeu de données

Le fichier `transactions.csv` contient 20 000 lignes et 8 colonnes. Chaque ligne décrit une transaction par trois identifiants, quatre montants et une date. Le tableau 1 présente les colonnes et le rôle de chacune dans l'analyse.

Table 1: Description des colonnes du jeu de données.

Colonne	Signification	Utilisée ?
global_id	Identifiant de la transaction	Non
sender_id	Identifiant de l'émetteur	Non
receiver_id	Identifiant du destinataire	Non
initial_amount	Solde initial du compte	**Oui**
transfer_amount	Montant transféré	**Oui**
amount_received	Montant réellement reçu	**Oui**
final_amount	Solde final du compte	**Oui**
date	Horodatage de la transaction	Non

Seuls les quatre montants sont retenus pour le clustering. Les identifiants sont de simples numéros sans signification métrique, et la date n'entre pas dans le calcul de distance entre transactions.

2.2 Hypothèses et conventions de départ

L'analyse repose sur un ensemble d'hypothèses explicites, qu'il convient de poser clairement car elles conditionnent l'interprétation des résultats.

- **Unité monétaire.** Tous les montants (`initial_amount`, `transfer_amount`, `amount_received`, `final_amount`) sont exprimés en euros.

- **Relations comptables attendues.** En situation normale, on suppose que le solde final découle des autres montants (`final_amount` $\approx$ `initial_amount` $-$ `transfer_amount`) et que l'argent reçu égale l'argent envoyé (`amount_received` $=$ `transfer_amount`). On suppose également que le solde transféré reste inférieur au solde disponible (`transfer_amount` $<$ `initial_amount`) et que tous les montants sont supérieurs à zéro (`amount_received`, `initial_amount`, `transfer_amount`, `final_amount`). Toute violation de ces relations constitue une incohérence métier.
- **Nombre de clusters.** La valeur $k = 6$ est fixée *a priori*.
- **Mesure de similarité.** La proximité entre deux transactions est mesurée par la distance euclidienne dans l'espace des montants normalisés ; deux transactions sont d'autant plus « semblables » que cette distance est faible.

2.3 Environnement technique

Le projet est réalisé en Python 3 avec les bibliothèques suivantes : **pandas** (manipulation des données), **NumPy** (calcul numérique), **scikit-learn** (StandardScaler, KMeans, PCA) et **matplotlib** (visualisations).

3. Méthodologie

Le traitement suit un *pipeline* en six étapes, chacune implémentée par une fonction dédiée.

1. **Chargement** des données depuis le fichier CSV.
2. **Sélection** des quatre caractéristiques numériques pertinentes.
3. **Normalisation** via `StandardScaler`.
4. **Clustering** par *K-means* en 6 groupes.
5. **Détection des anomalies** par distance au centroïde.
6. **Visualisation** après réduction de dimensions (PCA).

3.1 Pourquoi normaliser ?

Les montants couvrent des échelles très différentes : le solde initial atteint 375 000 € tandis que certains transferts ne valent que quelques euros. Or K-means raisonne en distances : sans mise à l'échelle, la variable `initial_amount` écraserait toutes les autres. Le `StandardScaler` centre chaque variable (moyenne 0) et la réduit (écart-type 1), plaçant ainsi les quatre montants sur un pied d'égalité.

Listing 1: Normalisation des caractéristiques.

```
from sklearn.preprocessing import StandardScaler

def normalize_features(df):
    scaler = StandardScaler()
    return scaler.fit_transform(df)  # moyenne 0, ecart-type 1
```

3.2 Détection des anomalies

Pour chaque transaction, on calcule la distance euclidienne à son centroïde. Sont déclarées anormales celles dont la distance dépasse un seuil fixé.

Listing 2: Détection des transactions éloignées de leur centre.

```python
import numpy as np

def detect_anomalies(data, model, threshold):
    labels = model.predict(data)
    centroids = model.cluster_centers_
    # distance de chaque point a SON propre centroide
    distances = np.linalg.norm(data - centroids[labels], axis=1)
    return np.where(distances > threshold)[0]
```

3.3 Choix automatique du seuil

La valeur du seuil est déterminante : trop basse, elle noie l'analyse sous les fausses alertes ; trop haute, elle laisse passer de vraies anomalies. Plutôt que de fixer une valeur arbitraire, on la calcule directement à partir de la distribution des distances. **Deux méthodes automatiques** sont présentées et comparées ci-dessous, avant de retenir la plus adaptée pour la suite du travail.

Première méthode : moyenne + 3 écarts-types

La première règle calcule le seuil à partir de la distribution des distances, selon la formule **moyenne + 3 écarts-types**.

Listing 3: Calcul du seuil par la règle des trois sigma.

```python
def suggest_threshold(data, model, k=3.0):
    labels = model.predict(data)
    centroids = model.cluster_centers_
    distances = np.linalg.norm(data - centroids[labels], axis=1)
    # seuil = moyenne + k ecarts-types de la distribution des distances
    return distances.mean() + k * distances.std()
```

Pourquoi « moyenne + 3 écarts-types » ?

Ce choix repose sur un raisonnement statistique classique, la **règle des trois sigma** ($\mu + 3\sigma$) :

- Pour une distribution approximativement normale, environ **99,7 %** des valeurs se situent en deçà de la moyenne plus trois écarts-types. Ne dépasser ce seuil que dans environ 0,3 % des cas garantit que seuls les points **réellement exceptionnels** sont retenus, et non la simple variabilité ordinaire.
- Le seuil est **adaptatif** : il se recalcule à partir des données. Si les transactions sont globalement plus dispersées, le seuil s'élève automatiquement ; il n'est pas figé sur une valeur qui conviendrait à un seul jeu de données.

- Il **équilibre sensibilité et précision** : la valeur $k = 3$ constitue un compromis éprouvé entre détecter trop (beaucoup de faux positifs, avec $k = 2$) et détecter trop peu (anomalies manquées, avec $k = 4$ ou plus).
- Il reste **ajustable** : le paramètre k permet de durcir ou d'assouplir la détection selon le niveau de vigilance souhaité.

Sur nos données, cette règle donne un seuil d'environ 1,22 (sur les distances normalisées) et retient **442 transactions**. À titre de comparaison, un seuil plus strict ($\mu + 4\sigma \approx 1{,}55$) n'en retiendrait que 219, tandis qu'un seuil plus permissif ($\mu + 2\sigma \approx 0{,}88$) en signalerait 1 159 : le réglage du paramètre k a donc un effet direct et maîtrisable sur le volume d'alertes.

Deuxième méthode : 98e percentile des distances

La seconde règle ne raisonne plus en écarts-types mais directement en **rang** : on fixe le seuil à la valeur en dessous de laquelle se trouvent 98 % des distances. Autrement dit, on déclare anormales les 2 % de transactions les plus éloignées de leur centre.

Listing 4: Calcul du seuil par le 98e percentile.

```
def suggest_threshold_percentile(data, model):
    distances = np.min(model.transform(data), axis=1)
    return np.percentile(distances, 98)
```

Pourquoi le 98e percentile ?

Ici, `model.transform(data)` renvoie la distance de chaque point à *tous* les centroïdes ; le `np.min(..., axis=1)` ne conserve que la distance au centroïde le plus proche qui, pour un point bien classé, est précisément son propre centroïde. La logique est la suivante :

- Le seuil cible une **proportion fixe**. Par construction, le 98e percentile laisse exactement 2 % des points au-dessus de lui. Le nombre d'anomalies est donc connu d'avance et stable : quelle que soit la forme exacte de la distribution, on isole toujours la même fraction extrême de la population.
- Il ne suppose **aucune forme de distribution**. Contrairement à la règle des trois sigma, qui s'appuie implicitement sur l'hypothèse de normalité, le percentile est purement **non paramétrique** : il lit le rang dans les données telles qu'elles sont. C'est un atout lorsque les distances sont fortement asymétriques, ce qui est le cas ici.
- Il est **robuste aux valeurs extrêmes**. Quelques distances gigantesques tirent fortement la moyenne et surtout l'écart-type vers le haut, ce qui peut gonfler artificiellement le seuil $\mu + 3\sigma$. Un percentile, lui, n'est pas sensible à l'amplitude des points extrêmes : seul leur *rang* compte.
- Il reste **ajustable** : remplacer 98 par 99 (top 1 %) ou 95 (top 5 %) durcit ou assouplit directement la détection, exactement comme le paramètre k de la première méthode.

Comparaison des deux méthodes. Le tableau 2 résume le volume d'anomalies produit par chaque règle.

Table 2: Nombre d'anomalies selon la méthode de seuillage retenue.

Méthode	Anomalies	% du total
$\mu + 2\sigma$ (permissif)	1 159	5,80 %
$\mu + 3\sigma$ (*1re méthode*)	442	2,21 %
98e percentile (*2e méthode*)	400	2,00 %
$\mu + 4\sigma$ (strict)	219	1,10 %

L'enseignement de cette comparaison est double. D'une part, les deux méthodes automatiques donnent des volumes très proches (442 contre 400), ce qui rassure sur la stabilité de la détection : autour de 2 % des transactions sont jugées atypiques quelle que soit l'approche. D'autre part, elles ne fonctionnent pas de la même façon. Le 98e percentile fixe le *nombre* d'anomalies (toujours 2 %) et laisse le seuil s'ajuster ; la règle $\mu + 3\sigma$ fixe le *seuil* (en unités d'écart-type) et laisse le nombre d'anomalies varier selon la dispersion réelle.

Méthode retenue pour la suite

Nous retenons la **deuxième méthode**, le **98e percentile des distances**, pour l'ensemble de l'analyse qui suit. Elle est en effet **non paramétrique** (elle ne suppose pas la normalité des distances, hypothèse clairement fausse ici vu la forte asymétrie) et **robuste aux valeurs extrêmes**, deux propriétés mieux adaptées à la nature très asymétrique de nos données que la règle des trois sigma. Avec ce choix, **400 transactions** (2,00 %) sont retenues comme anormales.

3.4 Réduction de dimensions par PCA

Les transactions vivent dans un espace à **quatre dimensions** (les quatre montants), impossible à représenter directement sur une feuille. Pour les visualiser, on emploie l'**analyse en composantes principales** (PCA, *Principal Component Analysis*).

Principe. La PCA cherche de nouveaux axes, appelés *composantes principales*, qui sont des combinaisons des variables d'origine. Le premier axe est orienté dans la direction où les données s'étirent le plus (variance maximale) ; le deuxième, perpendiculaire au premier, capte le maximum de la variance restante, et ainsi de suite. En ne conservant que les deux premières composantes, on obtient une « photographie » en 2D qui préserve l'essentiel de la structure des données.

Application à nos données. La projection retenue conserve une part remarquable de l'information : la première composante explique **93,1 %** de la variance et la seconde **6,9 %**, soit **100 %** à elles deux. Ce résultat n'est pas un hasard : il découle directement de la forte **corrélation entre les** variables observée à la section 4.3. Concrètement, **la vue 2D ne perd aucune information**, elle est une représentation fidèle, et non une approximation, des données réelles.

Comment l'interpréter ?

Sur le nuage projeté (figure 5), trois clés de lecture :

- La **proximité** traduit la ressemblance : deux points proches correspondent à des transactions aux montants similaires.
- L'**axe horizontal (PC1)** reflète surtout l'ampleur des montants, puisqu'il concentre 93 % de la variance : on se déplace des petites transactions (à gauche) vers les grosses (à droite).
- L'**éloignement** d'un point par rapport au cœur de son groupe signale son caractère atypique. C'est précisément ce que repèrent les anomalies en rouge.

4. Analyse exploratoire des données

Avant toute modélisation, une analyse exploratoire (EDA) permet de comprendre la structure des données et de valider les choix du pipeline.

4.1 Qualité des données

Le jeu de données est de très bonne qualité : aucune valeur manquante et aucun doublon sur les 20 000 lignes. Aucun nettoyage préalable n'est nécessaire.

4.2 Des distributions très asymétriques

Les quatre variables présentent une forte asymétrie (coefficient d'asymétrie proche de **2,0**) et une allure **bimodale** : une masse de petites transactions, puis une seconde bosse de très gros montants (figure 1). Cette coexistence de populations distinctes annonce la pertinence d'un clustering.

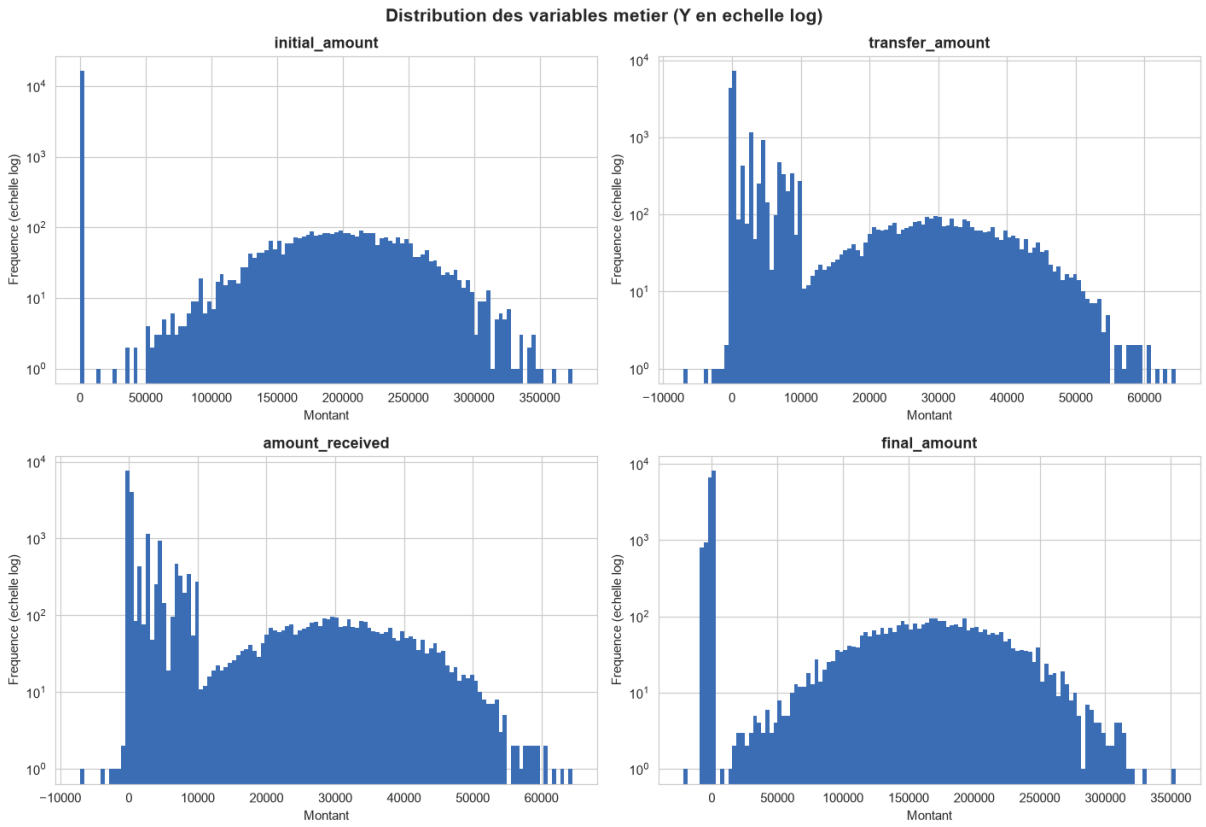


Figure 1: Distribution des quatre variables métier (axe vertical en échelle logarithmique). On distingue nettement deux populations : petites et grosses transactions.

La figure 2 confirme la présence massive de valeurs extrêmes (*outliers*, en rouge). C'est la conséquence directe de l'asymétrie et cela justifie pleinement l'étape de normalisation.

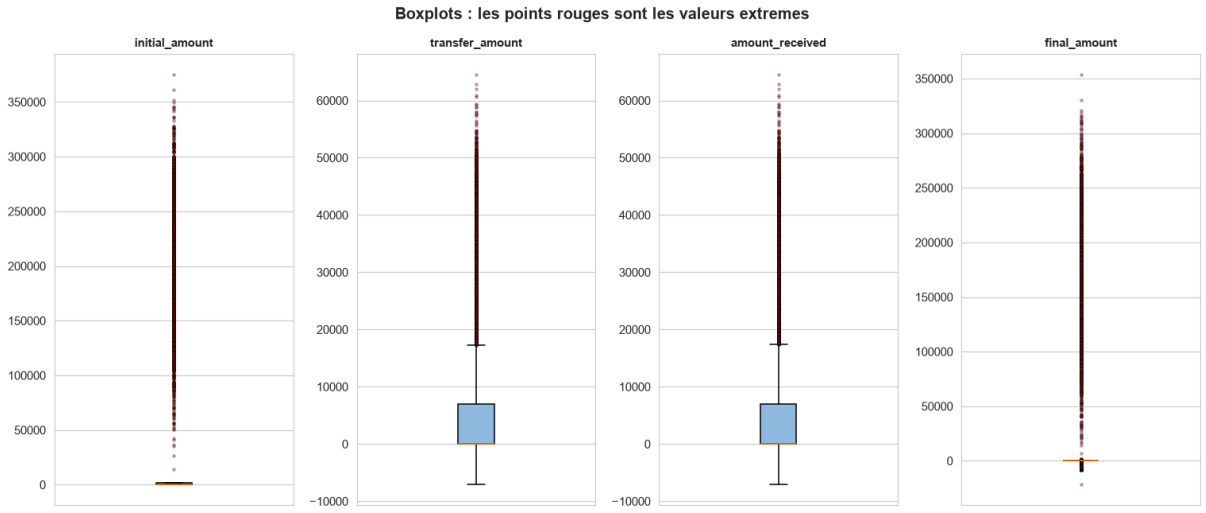


Figure 2: Boîtes à moustaches : les nombreux points rouges sont les valeurs extrêmes de chaque variable.

4.3 Forte corrélation entre variables

La matrice de corrélation (figure 3) révèle des liens très étroits : la corrélation entre montant transféré et montant reçu atteint **1,00**, et celle entre solde initial et solde final vaut **0,997**. Les quatre colonnes ne portent donc qu'environ deux dimensions d'information indépendantes. C'est pourquoi la projection PCA en deux dimensions, utilisée plus loin, conservera la **totalité** de l'information.

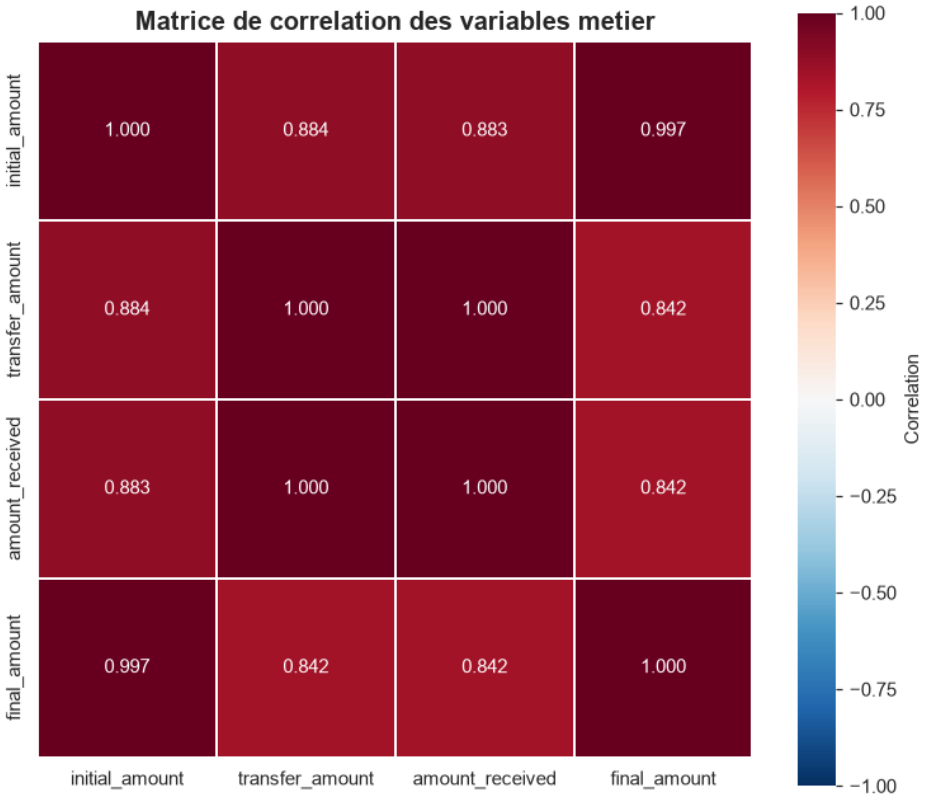


Figure 3: Matrice de corrélation des variables métier. Les corrélations proches de 1 traduisent une forte corrélation entre les colonnes.

4.4 Premiers comportements repérés

En appliquant quelques règles métier simples (antérieures au clustering), on distingue déjà plusieurs familles de transactions (figure 4) : les opérations **normales** (7 354 cas), les **découverts** au solde final négatif (4 355), les **transactions fantômes** où le montant reçu diffère du montant envoyé (4 289), les **sur-transferts** où l'on transfère plus que le solde disponible (3 904) et les rares **transferts négatifs** incohérents (98). Cette première lecture, fondée sur des règles, servira de point de comparaison pour interpréter les clusters trouvés par *K-means* et pour évaluer ce que la détection automatique capte réellement.

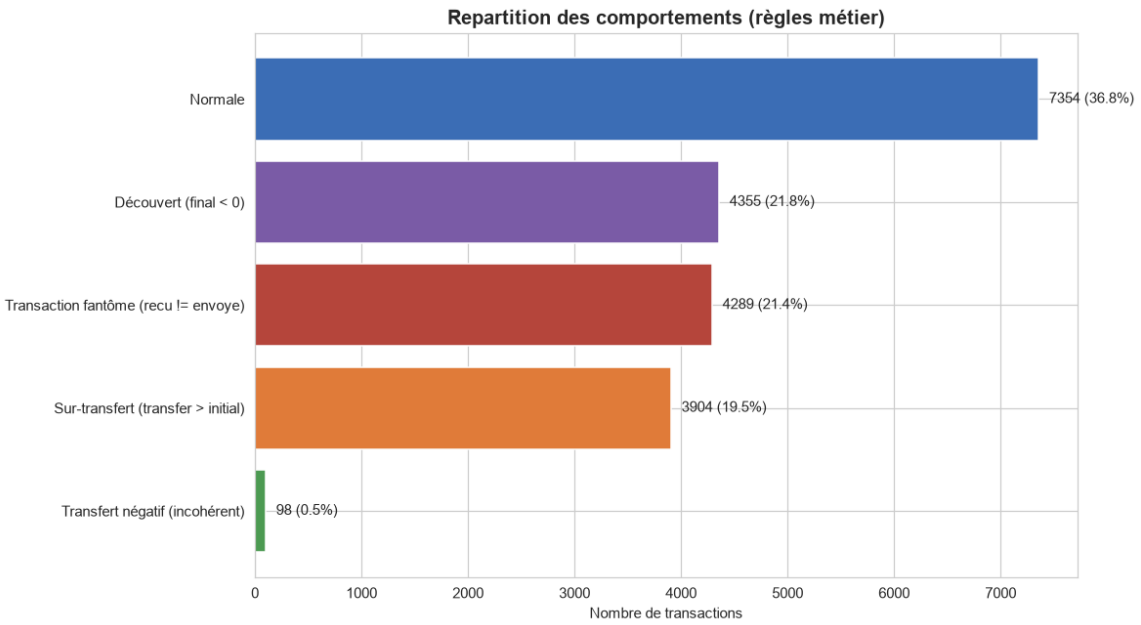


Figure 4: Répartition des comportements selon des règles métier simples (catégories indicatives, antérieures au clustering).

5. Résultats du clustering

5.1 Méthode d'interprétation

Un cluster ne possède pas d'étiquette : son comportement se déduit des **moyennes de ses caractéristiques**. Trois indicateurs suffisent :

- la **taille du compte** (valeur de `initial_amount`) ;
- l'**intensité des mouvements** (ratio `transfer_amount / initial_amount`) ; un ratio supérieur à 100 % signale un transfert plus grand que le solde disponible ;
- le **signe du solde final** : négatif révèle un découvert.

5.2 Profil des six clusters

Le tableau 3 présente, pour chaque cluster, les moyennes en valeurs réelles ainsi que le comportement déduit.

Table 3: Profil moyen des six clusters (valeurs réelles, en euros).

Cl.	Initial	Transfert	Final	Taille	Comportement
0	390	369	51	13 429 (67%)	Normal
2	2 256	6 431	−4 175	3 076 (15%)	Découvert (overdraft)
1	247 083	33 090	213 993	967 (5%)	Gros volume
3	153 776	28 464	125 312	985 (5%)	Gros volume
4	213 256	18 002	195 254	819 (4%)	Gros volume
5	182 312	43 558	138 754	724 (4%)	Gros volume actif

Comment retrouve-t-on les montants réels du tableau 3 ?

Les valeurs du tableau ci-dessus sont en euros, mais le modèle, lui, n'a jamais « vu » d'euros : il a été entraîné sur des données **normalisées** (moyenne 0, écart-type 1). Les centres de cluster qu'il fournit (`model.cluster_centers_`) vivent donc dans cet espace normalisé, et la projection de la figure 5 se situe encore un cran plus loin, dans l'espace PCA à deux dimensions. Pour revenir aux montants lisibles du tableau, il faut donc **remonter le pipeline à l'envers**, transformation par transformation. Deux situations se présentent.

Cas 1 : on part des centres de cluster (espace normalisé). C'est le chemin le plus direct, et celui qui produit le tableau 3. Le `StandardScaler` a appliqué à chaque montant la transformation $z = (x - \mu)/\sigma$. L'opération inverse, $x = z \cdot \sigma + \mu$, est fournie clé en main par `scaler.inverse_transform`. Il suffit de l'appliquer aux centroïdes :

Listing 5: Reconstruction des montants réels des centroïdes (tableau 3).

```python
import numpy as np
import pandas as pd

def cluster_profiles_in_euros(scaler, model, labels, features):
    """Retrouve les moyennes en euros de chaque cluster a partir
    des centroides exprimes en espace normalise."""
    # 1. Centres de cluster dans l'espace normalise (z-scores)
    centers_scaled = model.cluster_centers_           # shape (6, 4)
    # 2. Transformation inverse du StandardScaler : x = z * sigma + mu
    centers_euros = scaler.inverse_transform(centers_scaled)
    # 3. Mise en forme lisible
    profiles = pd.DataFrame(centers_euros, columns=features)
    # 4. Taille de chaque cluster
    profiles["taille"] = np.bincount(labels)
    return profiles.round(0)
```

Cas 2 : on part des coordonnées PCA (espace 2D de la figure 5). Si l'on ne dispose que des coordonnées projetées (par exemple pour annoter un point précis du nuage), il faut enchaîner **deux** inversions : d'abord remonter de la 2D PCA vers l'espace normalisé à quatre dimensions avec `pca.inverse_transform`, puis appliquer `scaler.inverse_transform` comme au cas 1. Attention : la PCA n'ayant conservé que deux composantes, son inversion est en théorie approximative; mais ici les deux axes captent 100 % de la variance (section 4.3), si bien que la reconstruction est exacte aux arrondis près.

Listing 6: Reconstruction des montants réels depuis l'espace PCA 2D.

```python
def pca_point_to_euros(coords_2d, pca, scaler, features):
    """coords_2d : array (n, 2) de coordonnees dans le plan PCA.
    Renvoie les montants reels correspondants, en euros."""
    # 1. PCA 2D  ->  espace normalise 4D
    data_scaled = pca.inverse_transform(coords_2d)    # shape (n, 4)
    # 2. Espace normalise 4D  ->  euros
    data_euros = scaler.inverse_transform(data_scaled)
    return pd.DataFrame(data_euros, columns=features).round(0)
```

En résumé, la chaîne directe est euros $\xrightarrow{\text{scaler}}$ normalisé $\xrightarrow{\text{PCA}}$ 2D, et il suffit de la parcourir en sens inverse, 2D $\xrightarrow{\text{pca.inverse_transform}}$ normalisé $\xrightarrow{\text{scaler.inverse_transform}}$ euros, pour récupérer les montants du tableau 3.

L'analyse fait ressortir **trois grands types de comportement**, que l'on détaille ci-dessous.

Cluster 0 — transactions normales (67 %). Ce groupe, de loin le plus peuplé avec 13 429 transactions, rassemble de petites opérations courantes : un solde initial moyen d'environ 390 €, un transfert de 369 € et un solde final positif de 51 €. Les montants sont faibles, cohérents entre eux (le reçu égale l'envoyé), et le compte ne bascule pas dans le rouge. C'est le **comportement de référence**, celui de l'immense majorité des clients dans leurs opérations quotidiennes. Sa forte densité explique la fine ligne compacte observée à gauche de la projection PCA (figure 5).

Cluster 2 — découverts / overdraft (15 %). Avec 3 076 transactions, ce cluster se distingue nettement par un **solde final négatif** (−4 175 € en moyenne). Le mécanisme est clair : le montant transféré (6 431 €) dépasse largement le solde initial disponible (2 256 €), soit un ratio transfert/solde de **285 %**. Autrement dit, ces comptes dépensent près de trois fois ce qu'ils possèdent, ce qui les fait basculer en situation de découvert. C'est un comportement à risque, parfaitement isolé par l'algorithme sans qu'on le lui ait indiqué.

Clusters 1, 3, 4 et 5 — gros volume (4 à 5 % chacun). Ces quatre groupes rassemblent les comptes à très gros montants, de 150 000 à 247 000 € de solde initial. Ils se ressemblent dans leur nature mais se différencient par l'ampleur et l'intensité des mouvements : le cluster 1 réunit les soldes les plus élevés (247 000 €), tandis que le cluster 5 se caractérise par les transferts les plus actifs (43 558 €, soit 24 % du solde). Ces quatre clusters **ne traduisent pas quatre comportements fondamentalement distincts**. K-means a simplement subdivisé la population continue des grosses transactions en plusieurs tranches de montant. C'est une conséquence directe du choix de fixer $k = 6$: l'algorithme doit produire six groupes, et il les trouve en découpant finement la zone la plus étalée de l'espace.

En synthèse, le clustering distingue donc essentiellement **trois régimes financiers** : le quotidien de faible montant (cluster 0), le découvert (cluster 2) et le gros volume (clusters 1, 3, 4, 5). Cette segmentation est cohérente avec la structure bimodale repérée dès l'analyse exploratoire.

6. Analyse des anomalies détectées

6.1 Combien d'anomalies ?

Avec la méthode retenue: le **98e percentile des distances** (section 3.3), le seuil isole exactement les 2 % de transactions les plus éloignées de leur centre, soit **400 transactions** (2,00 % de l'ensemble). C'est ce jeu de 400 anomalies qui sert de base à toute l'analyse ci-dessous.

6.2 À quels clusters appartiennent-elles ?

Les anomalies ne sont pas réparties uniformément (tableau 4) : elles se concentrent presque exclusivement dans les clusters de gros montant (1, 3, 4 et 5), qui totalisent 385 des 400 cas. Le cluster normal (cluster 0) n'en contient pratiquement aucune (4 sur 13 429).

Table 4: Répartition des anomalies par cluster (seuil au 98e percentile, 400 anomalies).

Cluster	Anomalies	Taille	% du cluster
0 — normales	4	13 429	0,0 %
2 — découverts	11	3 076	0,4 %
3 — gros volume	72	985	7,3 %
1 — très gros volume	87	967	9,0 %
5 — gros volume actif	105	724	14,5 %
4 — gros volume	121	819	14,8 %
TOTAL	**400**	**20 000**	**2,00 %**

Le montant initial moyen des anomalies atteint **197 246 €**, contre 35 458 € pour l'ensemble du jeu de données; soit près de six fois plus. Les anomalies détectées par distance sont donc avant tout des transactions de magnitude exceptionnelle. La figure 5 les montre en rouge, à la périphérie du grand nuage des opérations de gros montant : ce sont les points les plus éloignés du cœur de leur cluster, ceux dont les sommes en jeu s'écartent le plus de la moyenne de leur groupe.

Comment retrouve-t-on les montants réels du tableau 4 en partant du PCA ?

On veut afficher des montants en euros (le montant moyen, les effectifs et proportions par cluster), mais l'analyse vit dans des espaces transformés. Si l'on part des coordonnées du nuage PCA (figure 5), il faut enchaîner **les deux inversions** déjà décrites pour le tableau 3 : on remonte d'abord de la 2D PCA vers l'espace normalisé à quatre dimensions (`pca.inverse_transform`), puis de l'espace normalisé vers les euros (`scaler.inverse_transform`). C'est seulement *après* ce retour en euros que l'on calcule les statistiques affichées.

La fonction ci-dessous part donc directement des coordonnées PCA (`coords_2d`), reconstruit les montants réels et agrège les statistiques du tableau 4 :

Listing 7: Reconstruction des statistiques d'anomalies en euros, en partant du PCA (tableau 4).

```
def anomaly_stats_from_pca(coords_2d, labels, anomalies,
```

```
                              pca, scaler, features, n_clusters=6):
    """Part des coordonnees PCA 2D, remonte jusqu'aux euros, puis agrege
    par cluster les anomalies detectees.

    coords_2d : array (n, 2) -- coordonnees du nuage PCA (figure 5)
    """
    # 1. PCA 2D -> espace normalise 4D
    data_scaled = pca.inverse_transform(coords_2d)
    # 2. Espace normalise 4D -> euros
    data_euros = scaler.inverse_transform(data_scaled)
    df = pd.DataFrame(data_euros, columns=features)
    df["cluster"] = labels

    # 3. Marqueur booleen : la transaction est-elle une anomalie ?
    df["anomalie"] = False
    df.loc[anomalies, "anomalie"] = True

    # 4. Montant initial moyen des anomalies (~197 000 EUR)
    montant_moyen = df.loc[df["anomalie"], "initial_amount"].mean()

    # 5. Repartition des anomalies par cluster (effectif et %)
    par_cluster = (df[df["anomalie"]]
                   .groupby("cluster").size()
                   .reindex(range(n_clusters), fill_value=0))
    taille = df.groupby("cluster").size()
    pct = (par_cluster / taille * 100).round(1)
    return montant_moyen, par_cluster, pct
```

La clé est que la sélection des anomalies se fait dans l'espace des distances (indices `anomalies`), tandis que toute statistique *lisible* : montant moyen, effectifs, pourcentages, est calculée après reconstruction des euros depuis le plan PCA, par la chaîne `pca.inverse_transform` puis `scaler.inverse_transform`.

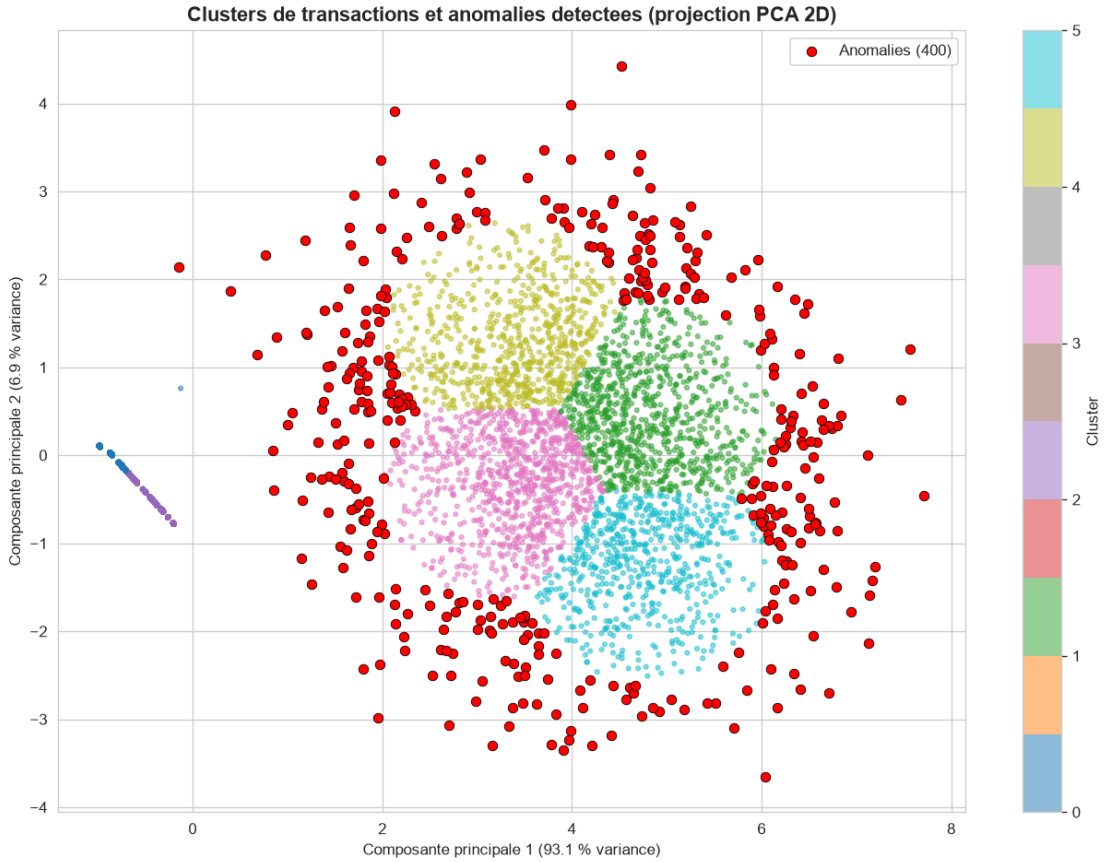


Figure 5: Projection PCA (2D) des transactions. Les couleurs distinguent les clusters ; les anomalies sont en rouge. À gauche, la fine ligne dense des petites transactions normales ; à droite, le nuage étalé des grosses opérations et ses points périphériques anormaux.

On notera la cohérence interne de ce résultat : plus un cluster regroupe des montants élevés et dispersés, plus il contient d'anomalies en proportion. Le cluster 4 culmine à 14,8 % d'anomalies, alors que le cluster 0 (petites transactions, très homogène) n'en compte quasiment aucune. La détection ne distribue donc pas les anomalies au hasard : elle suit fidèlement la dispersion de chaque groupe.

6.3 Que révèlent-elles ? Un constat essentiel

L'analyse de la *nature* des anomalies réserve une surprise instructive. Parmi les 400 anomalies, seules **7** présentent un transfert négatif et **aucune** ne correspond à une transaction fantôme. Pourtant, l'analyse exploratoire avait dénombré **4 309 transactions fantômes** et 98 transferts négatifs dans le jeu de données. Où sont donc passées ces incohérences manifestes ?

Le tableau 5 apporte la réponse : ces incohérences logiques sont **presque toutes concentrées dans le cluster 0**, celui des petites transactions normales.

Table 5: Localisation des incohérences métier par cluster.

Cluster	Transactions fantômes	Transferts négatifs
0 — normales	**4 309**	93
1 à 5 — autres	0	5

L'explication est géométrique. Ces transactions, bien qu'illogiques, portent sur de **petits montants**. Une fois les données normalisées, elles restent donc **très proches du centre du cluster 0**, où se masse l'essentiel des petites opérations. Leur distance au centroïde demeure faible, bien en deçà du seuil, quelle que soit la méthode de seuillage retenue et la détection les classe comme parfaitement normales. En d'autres termes, le défaut d'une transaction fantôme ne réside pas dans l'ampleur de ses montants (que mesure la distance) mais dans la relation entre eux.L'égalité attendue entre l'argent envoyé et l'argent reçu est une dimension que la distance euclidienne ne capture tout simplement pas.

Il en découle qu'il existe **deux natures d'anomalies** radicalement différentes dans ce jeu de données, et que la méthode n'en voit qu'une seule :

- les **anomalies de magnitude** — des montants anormalement élevés sont parfaitement détectées par la distance au centroïde ;
- les **anomalies logiques** — transactions fantômes, transferts négatifs, échappent totalement à la détection, car elles se dissimulent dans la masse dense des petites transactions normales.

Le point à retenir

La détection par distance répond à la question « ce montant est-il inhabituellement grand ? », mais pas à la question « cette transaction est-elle cohérente ? ». Les deux notions d'anomalie ne se recouvrent pas, et seule la première est captée ici et ce, indépendamment de la méthode de seuillage choisie.

7. Interprétation critique

La méthode K-means par distance au centroïde s'est révélée efficace pour deux tâches : segmenter la population en comportements interprétables et repérer les valeurs aberrantes par leur ampleur. Il importe néanmoins d'en discuter honnêtement les limites, car elles conditionnent l'usage qu'on peut faire des résultats.

7.1 Une détection aveugle aux incohérences logiques

La limite la plus marquante a déjà été établie : le modèle ne détecte pas les incohérences logiques de faible montant. Une transaction où l'argent envoyé diffère de l'argent reçu est manifestement suspecte, mais si les sommes en jeu sont petites, elle reste géométriquement proche de son centroïde et le modèle la classe comme normale. Sur ce jeu de données, ce sont tout de même 4 309 transactions fantômes qui passent ainsi inaperçues soit plus de 21 % du total. Une banque qui se reposerait uniquement sur cette méthode ignorerait donc une part très importante des

comportements problématiques.

7.2 Les causes profondes

Cette limite n'est pas un défaut de réglage mais découle de la nature même de l'approche, pour deux raisons :

- **La distance mesure l'amplitude, pas la cohérence.** La distance euclidienne au centroïde quantifie à quel point les montants d'une transaction s'écartent de la moyenne de son groupe. Elle ne sait rien des relations attendues entre ces montants (par exemple l'égalité envoyé = reçu, ou solde final = solde initial − transfert). Une violation de ces relations reste invisible tant qu'elle ne se traduit pas par un montant extrême.
- **K-means présuppose des clusters sphériques.** L'algorithme attribue chaque point au centroïde le plus proche, ce qui revient à supposer des groupes de forme grossièrement sphérique et de taille comparable. Cette hypothèse convient mal à des structures plus complexes ou à des densités très inégales, comme c'est le cas ici entre le cluster 0 (très dense) et les clusters de gros volume (très étalés).

7.3 Conséquence pour l'usage

En pratique, les 400 anomalies remontées par le modèle ne doivent pas être lues comme « les transactions frauduleuses », mais comme « les transactions dont les montants sont exceptionnellement élevés ». C'est une information utile, les très gros montants méritent une vérification mais partielle. Elle doit être complétée par d'autres signaux pour obtenir une couverture satisfaisante du risque, ce qui motive les recommandations de la section suivante.

8. Recommandations

1. **Compléter le clustering par des règles métier explicites** pour rattraper les anomalies logiques : signaler systématiquement les transactions où `amount_received` $\neq$ `transfer_amount` (fantômes) et où `transfer_amount` < 0 (transferts incohérents).
2. **Renforcer la surveillance des transactions de très gros montant** (clusters 1, 3, 4 et 5), où se concentre l'essentiel des anomalies géométriques.
3. **Conserver le 98e percentile comme règle de seuillage**, retenu ici pour sa robustesse et son caractère non paramétrique, tout en gardant la possibilité de l'ajuster (95e ou 99e percentile) selon le niveau de vigilance souhaité : un percentile plus bas augmente le nombre d'alertes (et de faux positifs), un percentile plus haut ne retient que les cas les plus extrêmes.
4. **Envisager des méthodes complémentaires** (Isolation Forest, DBSCAN) qui ne présupposent pas la forme sphérique des groupes et peuvent détecter des anomalies de structure plus fine.

9. Conclusion

L'application de K-means a permis de structurer 20 000 transactions en **six clusters interprétables**: un comportement normal dominant, une famille de découverts, quatre profils de gros volume et d'isoler, à l'aide du seuil au 98e percentile, **400 transactions atypiques** (2,00 %) par leur montant exceptionnel.

La démarche fournit une première grille de lecture précieuse des comportements financiers. Son principal enseignement est aussi sa principale limite : la détection par distance capte les extrêmes de magnitude mais laisse passer les incohérences logiques de faible montant. Ce constat plaide pour une **approche hybride**, associant la segmentation non supervisée à des règles métier ciblées.

10. Références

[1] F. Pedregosa et al., « Scikit-learn: Machine Learning in Python », *Journal of Machine Learning Research*, vol. 12, p. 2825–2830, 2011.

[2] S. Lloyd, « Least squares quantization in PCM », *IEEE Transactions on Information Theory*, vol. 28, n° 2, p. 129–137, 1982 (algorithme des k-moyennes).

[3] J. MacQueen, « Some methods for classification and analysis of multivariate observations », *Proc. 5th Berkeley Symposium on Mathematical Statistics and Probability*, p. 281–297, 1967.

[4] I. T. Jolliffe, *Principal Component Analysis*, 2e éd., Springer, 2002.

[5] F. Pukelsheim, « The Three Sigma Rule », *The American Statistician*, vol. 48, n° 2, p. 88–91, 1994.

[6] F. T. Liu, K. M. Ting, Z.-H. Zhou, « Isolation Forest », *IEEE International Conference on Data Mining (ICDM)*, p. 413–422, 2008.

[7] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, « A Density-Based Algorithm for Discovering Clusters (DBSCAN) », *Proc. KDD*, p. 226–231, 1996.

[8] Documentation scikit-learn : KMeans, StandardScaler et PCA. https://scikit-learn.org/stable/

A. Lancement du projet

Cette partie regroupe l'intégralité du code source du projet. Trois fichiers le constituent : le script d'analyse exploratoire (`EDA.py`), le pipeline de clustering (`k_means_pipeline.py`) et un script principal (`main.py`) qui orchestre les deux.

Les trois fichiers doivent être placés dans le même dossier, et le jeu de données `transactions.csv` dans un sous-dossier `data/` (le chemin attendu est `data/transactions.csv`). Après installation des dépendances, le script `main.py` enchaîne automatiquement les deux étapes :

Listing 8: Commandes de lancement.

```
# Installation des dependances
pip install pandas numpy scikit-learn matplotlib

# Lancer l'EDA puis le pipeline K-means (les deux a la suite)
python main.py

# Ou separement :
python main.py --eda       # uniquement l'analyse exploratoire
python main.py --kmeans    # uniquement le pipeline K-means
```

B. Script principal — main.py

Ce script utilise `argparse` pour importer les deux modules et exécuter leurs fonctions `main()` selon les options choisies.

Listing 9: main.py

```
"""
Point d'entree principal du projet.
Enchaine l'analyse exploratoire des donnees (EDA) puis le pipeline de
detection d'anomalies par K-means.

Utilisation :
    python main.py            -> lance l'EDA puis le pipeline K-means
    python main.py --eda      -> lance uniquement l'EDA
    python main.py --kmeans   -> lance uniquement le pipeline K-means
"""

import argparse

import EDA
import k_means_pipeline


def run_eda():
    print("\n" + "#" * 70)
    print("# ETAPE 1 : ANALYSE EXPLORATOIRE DES DONNEES (EDA)")
    print("#" * 70 + "\n")
    EDA.main()


def run_kmeans():
    print("\n" + "#" * 70)
```

```
    print("# ETAPE 2 : PIPELINE K-MEANS - DETECTION D'ANOMALIES")
    print("#" * 70 + "\n")
    k_means_pipeline.main()


def main():
    parser = argparse.ArgumentParser(
        description="Lance l'EDA et/ou le pipeline K-means de detection d'
            anomalies."
    )
    parser.add_argument("--eda", action="store_true", help="Lance uniquement l'
        EDA")
    parser.add_argument("--kmeans", action="store_true", help="Lance uniquement
        le pipeline K-means")
    args = parser.parse_args()

    if not args.eda and not args.kmeans:
        run_eda()
        run_kmeans()
    elif args.eda:
        run_eda()
    elif args.kmeans:
        run_kmeans()


if __name__ == "__main__":
    main()
```

C. Analyse exploratoire — EDA.py

Listing 10: EDA.py

```
"""
Analyse Exploratoire des Donnees (EDA).

Objectif : comprendre le jeu de donnees AVANT toute modelisation. On verifie
la qualite des donnees, on etudie les distributions, on detecte les valeurs
extremes, on mesure les correlations et on identifie les comportements metier.
Ces constats justifient ensuite les choix du pipeline K-means.
"""

import os
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")          # backend sans interface (sauvegarde fichier)
import matplotlib.pyplot as plt
import seaborn as sns


# Configuration
CSV_PATH = "data/transactions.csv"
FIG_DIR = "figures"
FEATURES = ["initial_amount", "transfer_amount",
            "amount_received", "final_amount"]
```

```
24
25 sns.set_style("whitegrid")
26 pd.set_option("display.width", 120)
27 pd.set_option("display.max_columns", 20)
28
29
30 # 1. Chargement
31 def load_data(path: str) -> pd.DataFrame:
32     """Charge le CSV et renvoie un DataFrame."""
33     return pd.read_csv(path)
34
35
36 # 2. Vue d'ensemble : qualite des donnees
37 def overview(df: pd.DataFrame) -> None:
38     print("=" * 70)
39     print("1. VUE D'ENSEMBLE")
40     print("=" * 70)
41     print(f"Dimensions : {df.shape[0]} lignes x {df.shape[1]} colonnes")
42     print(f"Colonnes   : {list(df.columns)}")
43     print(f"Doublons (lignes)       : {df.duplicated().sum()}")
44     print(f"Valeurs manquantes total: {df.isnull().sum().sum()}")
45     print("\nApercu des 5 premieres lignes :")
46     print(df.head().to_string())
47     print()
48
49
50 # 3. Statistiques descriptives
51 def descriptive_stats(df: pd.DataFrame) -> None:
52     print("=" * 70)
53     print("2. STATISTIQUES DESCRIPTIVES (variables metier)")
54     print("=" * 70)
55     desc = df[FEATURES].describe(percentiles=[.01, .25, .5, .75, .95, .99]).T
56     print(desc[["mean", "std", "min", "50%", "95%", "99%", "max"]].to_string())
57     print()
58
59
60 # 4. Coherence metier et signaux d'anomalie
61 def business_logic(df: pd.DataFrame) -> None:
62     """Verifie les relations comptables attendues et les incoherences."""
63     print("=" * 70)
64     print("3. COHERENCE METIER ET SIGNAUX D'ANOMALIE")
65     print("=" * 70)
66
67     # Regle : final_amount = initial_amount - transfer_amount
68     ecart_compta = df["final_amount"] - (df["initial_amount"] - df["
           transfer_amount"])
69     print("a) final_amount == initial_amount - transfer_amount ?")
70     print(f"   coherentes (ecart ~ 0) : {(ecart_compta.abs() < 0.01).sum()}")
71     print(f"   avec ecart             : {(ecart_compta.abs() >= 0.01).sum()}")
72
73     # Argent recu == argent envoye ?
74     ecart_recu = df["amount_received"] - df["transfer_amount"]
75     print("\nb) amount_received == transfer_amount ?")
76     print(f"   egaux                  : {(ecart_recu.abs() < 0.01).sum()}")
77     print(f"   perte (recu < envoye): {(ecart_recu < -0.01).sum()}")
78
79     # Regle : final_amount < initial_amount
80     print("\nc) final_amount < initial_amount ?")
```

```
    print(f"   respectee             : {(df['final_amount'] < df['initial_amount'
        ']).sum()}")
    print(f"   violee                : {(df['final_amount'] >= df['initial_amount
        ']).sum()}")

    # Regle : tous les montants strictement positifs
    pos = (df[FEATURES] > 0).all(axis=1)
    print("\nd) tous les montants > 0 ?")
    print(f"   tous positifs         : {pos.sum()}")
    print(f"   au moins un negatif   : {(~pos).sum()}")

    # Signaux d'anomalie bruts
    print("\ne) Signaux d'incoherence (regles metier) :")
    print(f"   transfer_amount negatif      : {(df['transfer_amount'] < 0).sum()
        }")
    print(f"   final_amount negatif (decouvert): {(df['final_amount'] < 0).sum()
        }")
    print(f"   transfer > initial (sur-transfert): {(df['transfer_amount'] > df
        ['initial_amount']).sum()}")
    print()


# 5. Categorisation des comportements (regles metier)
def categorize_behaviors(df: pd.DataFrame) -> pd.Series:
    """Attribue une categorie metier a chaque transaction selon des regles
    de priorite. Sert de point de comparaison pour les clusters K-means."""
    def rule(row):
        if row["transfer_amount"] < 0:
            return "Transfert negatif (incoherent)"
        if row["final_amount"] < 0:
            return "Decouvert (final < 0)"
        if abs(row["amount_received"] - row["transfer_amount"]) > 0.01:
            return "Transaction fantome (recu != envoye)"
        if row["transfer_amount"] > row["initial_amount"]:
            return "Sur-transfert (transfer > initial)"
        return "Normale"

    cat = df.apply(rule, axis=1)
    print("=" * 70)
    print("4. REPARTITION DES COMPORTEMENTS (regles metier)")
    print("=" * 70)
    for name, n in cat.value_counts().items():
        print(f"   {name:<40} {n:>6} ({n / len(df) * 100:5.1f} %)")
    print()
    return cat


# 6. Correlations
def correlations(df: pd.DataFrame) -> pd.DataFrame:
    print("=" * 70)
    print("5. MATRICE DE CORRELATION")
    print("=" * 70)
    corr = df[FEATURES].corr()
    print(corr.round(3).to_string())
    return corr


# 7. Visualisations
```

```python
def plot_distributions(df: pd.DataFrame) -> None:
    fig, axes = plt.subplots(2, 2, figsize=(13, 9))
    for ax, c in zip(axes.ravel(), FEATURES):
        ax.hist(df[c], bins=120, color="#3b6db5", edgecolor="none")
        ax.set_title(c, fontsize=12, fontweight="bold")
        ax.set_yscale("log")
        ax.set_xlabel("Montant")
        ax.set_ylabel("Frequence (echelle log)")
    fig.suptitle("Distribution des variables metier (Y en echelle log)",
                 fontsize=14, fontweight="bold")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig1_distributions.png"),
                dpi=110, bbox_inches="tight")
    plt.close()


def plot_boxplots(df: pd.DataFrame) -> None:
    fig, axes = plt.subplots(1, 4, figsize=(14, 6))
    for ax, c in zip(axes, FEATURES):
        ax.boxplot(df[c], patch_artist=True,
                   boxprops=dict(facecolor="#8fb8de"),
                   flierprops=dict(marker="o", markersize=2, alpha=0.3,
                                   markerfacecolor="red"))
        ax.set_title(c, fontsize=10, fontweight="bold")
        ax.set_xticks([])
    fig.suptitle("Boxplots : les points rouges sont les valeurs extremes",
                 fontsize=13, fontweight="bold")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig2_boxplots.png"),
                dpi=110, bbox_inches="tight")
    plt.close()


def plot_correlation(corr: pd.DataFrame) -> None:
    fig, ax = plt.subplots(figsize=(7, 6))
    sns.heatmap(corr, annot=True, fmt=".3f", cmap="RdBu_r", center=0,
                vmin=-1, vmax=1, square=True, linewidths=1, ax=ax,
                cbar_kws={"label": "Correlation"})
    ax.set_title("Matrice de correlation des variables metier",
                 fontsize=13, fontweight="bold")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig3_correlation.png"),
                dpi=110, bbox_inches="tight")
    plt.close()


def plot_categories(cat: pd.Series, n_total: int) -> None:
    counts = cat.value_counts()
    colors = ["#4c9a52", "#e07b39", "#b5453b", "#7a5ba6", "#3b6db5"]
    fig, ax = plt.subplots(figsize=(11, 6))
    bars = ax.barh(counts.index[::-1], counts.values[::-1],
                   color=colors[:len(counts)])
    for bar in bars:
        w = bar.get_width()
        ax.text(w + n_total * 0.005, bar.get_y() + bar.get_height() / 2,
                f"{int(w)} ({w / n_total * 100:.1f}%)", va="center", fontsize
                    =10)
    ax.set_xlabel("Nombre de transactions")
```

```
    ax.set_title("Repartition des comportements (regles metier)",
                 fontsize=13, fontweight="bold")
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "fig4_categories.png"),
                dpi=110, bbox_inches="tight")
    plt.close()


# Point d'entree
def main():
    os.makedirs(FIG_DIR, exist_ok=True)
    df = load_data(CSV_PATH)

    overview(df)
    descriptive_stats(df)
    business_logic(df)
    cat = categorize_behaviors(df)
    corr = correlations(df)

    plot_distributions(df)
    plot_boxplots(df)
    plot_correlation(corr)
    plot_categories(cat, len(df))

    print("=" * 70)
    print(f"Figures enregistrees dans le dossier '{FIG_DIR}/'")
    print("=" * 70)


if __name__ == "__main__":
    main()
```

D. Pipeline K-means — k_means_pipeline.py

Listing 11: k_means_pipeline.py

```
"""
Pipeline de detection d'anomalies par K-means.

Enchaine les etapes du projet :
    1. Chargement des donnees       -> load_data
    2. Selection des caracteristiques -> select_features
    3. Normalisation                -> normalize_features
    4. Clustering K-means           -> apply_kmeans
    5. Detection des anomalies      -> detect_anomalies
    6. Visualisation                -> plot_clusters
    7. Script principal             -> main
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

```python
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA


# Configuration
CSV_PATH = "data/transactions.csv"
N_CLUSTERS = 6
FIG_DIR = "figures"
RANDOM_STATE = 42
FEATURES = ["initial_amount", "transfer_amount",
            "amount_received", "final_amount"]


# Etape 1 : Chargement des donnees
def load_data(file_path: str) -> pd.DataFrame:
    df = pd.read_csv(file_path)
    print(f"[1] Donnees chargees : {df.shape[0]} lignes, {df.shape[1]} colonnes")
    return df


# Etape 2 : Selection des caracteristiques
def select_features(df: pd.DataFrame) -> pd.DataFrame:
    selected = df[FEATURES].copy()
    print(f"[2] Caracteristiques retenues : {FEATURES}")
    return selected


# Etape 3 : Normalisation
def normalize_features(df: pd.DataFrame):
    scaler = StandardScaler()
    data = scaler.fit_transform(df)
    print(f"[3] Donnees normalisees (moyenne~0, ecart-type~1) : shape {data.shape}")
    return data, scaler          # on renvoie aussi le scaler pour l'inversion


# Etape 4 : Clustering K-means
def apply_kmeans(data: np.ndarray, n_clusters: int = 6) -> tuple:
    model = KMeans(n_clusters=n_clusters, n_init=10, random_state=RANDOM_STATE)
    labels = model.fit_predict(data)
    print(f"[4] K-means applique avec {n_clusters} clusters "
          f"(inertie = {model.inertia_:.1f})")
    return model, labels


# Etape 5 : Detection des anomalies
def detect_anomalies(data: np.ndarray, model: KMeans, threshold: float) -> np.ndarray:
    labels = model.predict(data)
    centroids = model.cluster_centers_
    distances = np.linalg.norm(data - centroids[labels], axis=1)
    anomalies = np.where(distances > threshold)[0]
    print(f"[5] Seuil = {threshold:.3f} -> {len(anomalies)} anomalies "
          f"({len(anomalies) / len(data) * 100:.2f} % du total)")
    return anomalies


```

```python
# --- Methode 1 : moyenne + k ecarts-types (presentee a titre de comparaison) ---
def suggest_threshold(data: np.ndarray, model: KMeans, k: float = 3.0) -> float:
    """Seuil = moyenne + k * ecart-type des distances aux centroides."""
    labels = model.predict(data)
    distances = np.linalg.norm(data - model.cluster_centers_[labels], axis=1)
    return distances.mean() + k * distances.std()


# --- Methode 2 : 98e percentile des distances (METHODE RETENUE) ---
def suggest_threshold_percentile(data, model):
    """Seuil = 98e percentile des distances au centroide le plus proche.
    Isole par construction les 2 % de transactions les plus eloignees.
    C'est la methode retenue pour le projet (non parametrique, robuste)."""
    distances = np.min(model.transform(data), axis=1)
    return np.percentile(distances, 98)


# --- Reconstruction 1 : profils des clusters en euros (tableau 3) ---
def cluster_profiles_in_euros(scaler, model, labels):
    """Retrouve les moyennes en euros de chaque cluster a partir
    des centroides exprimes en espace normalise."""
    centers_euros = scaler.inverse_transform(model.cluster_centers_)
    profiles = pd.DataFrame(centers_euros, columns=FEATURES)
    profiles["taille"] = np.bincount(labels)
    return profiles.round(0)


# --- Reconstruction 2 : depuis le plan PCA 2D vers les euros ---
def pca_point_to_euros(coords_2d, pca, scaler, features=FEATURES):
    """coords_2d : array (n, 2) de coordonnees dans le plan PCA.
    Renvoie les montants reels correspondants, en euros.
    Enchaine deux inversions : PCA 2D -> normalise 4D -> euros."""
    # 1. PCA 2D -> espace normalise 4D
    data_scaled = pca.inverse_transform(coords_2d)
    # 2. Espace normalise 4D -> euros
    data_euros = scaler.inverse_transform(data_scaled)
    return pd.DataFrame(data_euros, columns=features).round(0)


# --- Reconstruction 3 : statistiques d'anomalies en euros, depuis le PCA (tableau 4) ---
def anomaly_stats_from_pca(coords_2d, labels, anomalies, pca, scaler,
                           features=FEATURES, n_clusters=N_CLUSTERS):
    """Part des coordonnees PCA 2D, remonte jusqu'aux euros, puis agrege
    par cluster les anomalies detectees (montant moyen, effectifs, %)."""
    # 1. PCA 2D -> normalise 4D -> euros
    data_euros = pca_point_to_euros(coords_2d, pca, scaler, features)
    df = data_euros.copy()
    df["cluster"] = labels
    df["anomalie"] = False
    df.loc[anomalies, "anomalie"] = True
    # 2. Montant initial moyen des anomalies (~197 000 EUR)
    montant_moyen = df.loc[df["anomalie"], "initial_amount"].mean()
    # 3. Repartition des anomalies par cluster (effectif et %)
    par_cluster = (df[df["anomalie"]]
                   .groupby("cluster").size()
                   .reindex(range(n_clusters), fill_value=0))
```

```
    taille = df.groupby("cluster").size()
    pct = (par_cluster / taille * 100).round(1)
    return montant_moyen, par_cluster, pct


# Etape 6 : Visualisation
def plot_clusters(data: np.ndarray, labels: np.ndarray, anomalies: np.ndarray):
    pca = PCA(n_components=2, random_state=RANDOM_STATE)
    coords = pca.fit_transform(data)
    var = pca.explained_variance_ratio_

    plt.figure(figsize=(11, 8))
    scatter = plt.scatter(coords[:, 0], coords[:, 1], c=labels,
                          cmap="tab10", s=8, alpha=0.5)
    plt.scatter(coords[anomalies, 0], coords[anomalies, 1],
                c="red", s=40, edgecolors="black", linewidths=0.5,
                label=f"Anomalies ({len(anomalies)})", zorder=5)
    plt.colorbar(scatter, label="Cluster")
    plt.xlabel(f"Composante principale 1 ({var[0]*100:.1f} % variance)")
    plt.ylabel(f"Composante principale 2 ({var[1]*100:.1f} % variance)")
    plt.title("Clusters de transactions et anomalies detectees (projection PCA 2
        D)",
              fontsize=13, fontweight="bold")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(FIG_DIR, "clusters_anomalies.png"),
                dpi=120, bbox_inches="tight")
    plt.close()
    print(f"[6] Graphique enregistre (2 axes PCA = {var.sum()*100:.1f} % de
        variance)")
    # On renvoie l'objet PCA et les coordonnees pour les reconstructions
    return pca, coords


# Etape 7 : Script principal
def main():
    df = load_data(CSV_PATH)
    features = select_features(df)
    data, scaler = normalize_features(features)
    model, labels = apply_kmeans(data, n_clusters=N_CLUSTERS)

    # Comparaison des deux methodes de seuillage
    t_sigma = suggest_threshold(data, model, k=3.0)
    t_pct = suggest_threshold_percentile(data, model)
    print(f"    Seuil mu+3sigma   = {t_sigma:.3f}  (1re methode, comparaison)")
    print(f"    Seuil 98e pctile  = {t_pct:.3f}  (2e methode, RETENUE)")

    # On retient la 2e methode : le 98e percentile
    anomalies = detect_anomalies(data, model, t_pct)
    pca, coords = plot_clusters(data, labels, anomalies)

    # Tableau 3 : profils des clusters en euros
    print(cluster_profiles_in_euros(scaler, model, labels))

    # Tableau 4 : statistiques d'anomalies en euros, reconstruites depuis le PCA
    montant_moyen, par_cluster, pct = anomaly_stats_from_pca(
        coords, labels, anomalies, pca, scaler)
    print(f"Montant initial moyen des anomalies : {montant_moyen:.0f} EUR")
```

```
187     print(par_cluster)
188     print(pct)
189 
190     return df, data, scaler, model, labels, anomalies
191 
192 
193 if __name__ == "__main__":
194     main()
```